

From Trained Model to Live Fraud Detection System

Zidio Development

Week Three & Week Four Report:- Model Training, Evaluation, and Deployment

Abstract

This report picks up exactly where the Week 2 report left off. At the end of Week 2, we had two things sitting on disk: a balanced training set (where fraud and normal transactions were represented equally, thanks to SMOTE) and a completely untouched, honest test set. Week 3 is where those files finally get used for something training two different machine learning models and grading them honestly against data they'd never seen before. Week 4 is where those trained models stop being notebook experiments and become a real, callable system: a live API that a payment terminal could talk to, an alerting pipeline that emails a human the moment something looks wrong, and an early architecture sketch for handling real-time transaction floods at scale. Just like the Week 2 report, this one goes through every single cell and every script involved, answering the same questions each time: what is this code actually doing, why are we doing it, why are we doing it at this exact point in the story, what did the output actually turn out to be, what does that output mean for the business, and what comes next. Nothing has been skipped including the honest, slightly humbling numbers from the model evaluation, the live terminal run of the actual API, and the real email alert it triggered.

Introduction: Where We Left Off, and Where We're Going

Quick recap of the story so far. Week 1 got clean transaction data out of a database. Week 2 was spent understanding that data discovering that fraud is rare (just 1.1% of everything), that fraud tends to involve bigger amounts, that it clusters at certain hours of the day, and that age alone doesn't say much. Week 2 ended by carefully splitting the data into a training portion and a testing portion, and using SMOTE to balance only the training portion, so the model would have enough fraud examples to actually learn from, while the test portion stayed completely real and untouched for an honest evaluation later.

Week 3 is that “later.” This is where we actually train models on the balanced training data and then check, honestly, how well they perform against the untouched test set. We don't train one model and call it done, we train two, because they catch fraud in two completely different ways, and Week 3 explains exactly why that matters.

Week 4 is the payoff. A trained model sitting in a notebook doesn't protect anyone, it has to be wired into something that can receive a real transaction the moment it happens and make a split-second decision. Week 4 builds that: a live web API, a rule that combines both models' opinions into one final decision, and a real email alert system through AWS. Everything in this report is backed by the actual notebook outputs, the actual saved model files, and for Week 4 an actual terminal run and a real email that landed in an inbox, which we'll walk through in detail.

WEEK 3: Model Training and Evaluation

The notebook for this week is called **03_Model_Training_and_Evaluation**, and it has nine cells. Every single cell in this notebook already includes its own built-in explanation as comments (what the code is about, why we're doing it, the intent behind it, and what result to expect) which is great practice, and this report expands on that same structure, cell by cell, adding the actual numbers that came back and what they genuinely mean.

Step 1: Loading the Balanced Training Data

What is this code?

This cell imports Pandas and loads the four CSV files that Week 2 produced: `X_train_balanced.csv` and `y_train_balanced.csv` (the SMOTE-balanced training data), and `X_test_final.csv` and `y_test_final.csv` (the untouched test data). It then runs `.values.ravel()` on both `y_train` and `y_test`, this simply flattens what Pandas loads as a two-dimensional, single-column table into a plain one-dimensional array, because that's the shape Scikit-Learn's models expect a target variable to be in.

Why are we doing this, and why now?

This has to be the very first cell of Week 3, for a simple reason: training a model on data that isn't loaded yet is impossible. But there's a deeper reason it's structured this way, by loading from saved CSV files instead of re-running Week 1 and Week 2's entire cleaning and balancing pipeline from scratch, we get a clean, reliable, isolated starting point. It also completely rules out a certain category of mistake: there is no way test data can accidentally leak into the training data at this stage, because the two were already separated and saved as different files before this notebook even opened.

What did we get, and what does it actually confirm?

The printed shapes were Training Features Shape: (158236, 29) and Testing Features Shape: (20000, 29). Digging into the actual files behind those numbers tells an important story: the training set holds exactly 79,118 normal transactions and 79,118 fraud transactions, a perfect, deliberate 50/50 balance from SMOTE, while the test set holds 19,779 normal transactions against just 221 fraud transactions, which is almost exactly the real-world 98.9%/1.1% split we discovered back in Week 2. That's the whole point working correctly: the model gets to train on a fair fight between the two classes, but gets graded on a test set that reflects reality, not a rigged one.

One honest detail worth flagging for the team: this particular saved training set keeps all 29 numeric columns exactly as they were in the original table including identifier-style numeric fields like `cc_num`, `zip`, and `acct_num`, rather than the version discussed at the very end of the Week 2 report that deliberately dropped `cc_num` and `zip` first. It's not a problem by itself, but it does mean the model technically had access to those ID-like columns during training. We'll come back to this directly in Step 5, when we look at feature importance, to check whether the model actually leaned on them.

Step 2: Training the Random Forest Classifier

What is this code?

This cell imports RandomForestClassifier from Scikit-Learn, creates a new one with `model = RandomForestClassifier(random_state=42)`, and then calls `model.fit(X_train, y_train)` to actually train it on the 158,236-row balanced training set.

Why are we doing this, and why now?

This is the moment the project stops being pure data preparation and becomes actual machine learning. A Random Forest works by building a large number of individual decision trees (100 of them, by default, since `n_estimators` wasn't manually overridden here), each one trained on a slightly different random sample of the training data and a random subset of the features. When it's time to make a prediction, all 100 trees vote, and the majority answer wins. This “wisdom of the crowd” approach tends to be far more reliable than any single decision tree, and it's also easy to explain to non-technical stakeholders, which matters for a fraud system that will eventually need business sign-off. `random_state=42` is set purely so that anyone on the team who reruns this cell gets the exact same trained model back, rather than a slightly different one each time.

What did we get?

The console simply printed “Success! The model has been trained.” Behind that one quiet line, 100 decision trees were actually built and fitted against the balanced data. At this point, the model object in memory has gone from a completely blank, untrained algorithm to a fully functional fraud-detection engine it just hasn't been tested against anything real yet.

What happens next?

The very next cell finally answers the only question that actually matters: how good is this model when it's shown transactions it has never seen before?

Step 3: Predictions and Evaluation The Confusion Matrix and Classification Report

What is this code?

This cell asks the freshly trained model to predict on `X_test`, the 20,000 test transactions it has never encountered in any form, producing `y_pred`. It then compares those predictions against the real answers, `y_test`, using two tools: a confusion matrix (a 2x2 grid of exactly how many predictions were right or wrong, and in which direction) and a classification report (a full breakdown of precision, recall, and F1-score for each class).

Why are we doing this, and why now?

This is the single most important cell in Week 3, because plain accuracy is genuinely useless for a problem like this a model that always guesses “normal” would score 98.9% accuracy while catching zero fraud. What actually matters is Recall (out of all the real fraud that happened, how much did we actually catch?) and Precision (out of everything we flagged as fraud, how often were we actually right?). This

cell exists specifically to surface those two numbers honestly, rather than letting a misleadingly high accuracy score hide the real story.

What did the numbers actually turn out to be, and what do they mean?

The confusion matrix came back as [[19556, 223], [125, 96]]. Reading that grid in plain terms: 19,556 normal transactions were correctly left alone, 223 normal transactions were wrongly flagged as fraud (false alarms), 125 real fraud cases slipped through completely undetected and were left marked as normal, and 96 real fraud cases were correctly caught.

Class	Precision	Recall	F1-score	Support
Normal (0)	0.99	0.99	0.99	19,779
Fraud (1)	0.30	0.43	0.36	221
Accuracy			0.98	20,000
Macro avg	0.65	0.71	0.67	20,000
Weighted avg	0.99	0.98	0.98	20,000

Here's the honest read of that table, exactly the way it should be explained to the team. Overall accuracy sits at a shiny-looking 98%, and if we stopped reading right there, it would look like a fantastic model this is the exact “Accuracy Paradox” trap Week 2 warned about, showing up right on schedule. Look at the fraud row instead: recall is 0.43, meaning the model only caught 43% of the actual fraud in the test set, 96 out of 221 real cases and let the other 57% (125 cases) walk straight through undetected. Precision on fraud is 0.30, meaning that whenever the model does raise its hand and say “fraud,” it's only correct 3 times out of 10; the other 7 times, it's wrongly accusing an innocent, legitimate transaction. In plain business terms: this first version of the model is a useful but genuinely imperfect fraud catcher, not a finished product, it misses more fraud than it catches, and when it does flag something, it's wrong more often than it's right. That's not a failure of the project; it's an honest, necessary finding, and exactly the kind of result that justifies bringing in a second, differently-built model, which is exactly what Step 9 does.

Step 4: Visualizing the Confusion Matrix on a Logarithmic Scale

What is this code?

This cell rebuilds the same confusion matrix from Step 3, but this time draws it as a color-coded heatmap using Seaborn, with one deliberate addition: `norm=LogNorm()`, which forces the color scale to follow a logarithmic curve instead of a straight linear one.

Why are we doing this, and why now?

Look at the four numbers again: 19,556 versus 223, 125, and 96. On an ordinary linear color scale, that first number is so enormous compared to the other three that all three smaller boxes would end up looking like nearly the same, barely-visible pale color you'd lose the ability to visually tell 223 apart from 96 at a

glance. A logarithmic color scale compresses that huge range so that every one of the four boxes gets its own genuinely distinct color, which matters enormously when this chart eventually ends up in front of stakeholders who need to understand the error pattern in three seconds, not three minutes.

What does this chart actually show?

The top-left box (19,556, correctly identified normal transactions) is a deep purple, by far the dominant block. The top-right box (223 false alarms) sits in orange. The bottom-left box (125 missed fraud cases) is a rose-red. And the bottom-right box (96 correctly caught fraud cases) is the deepest maroon-red on the whole grid. Visually, this makes an important point instantly obvious: the bottom row (the actual fraud row) is where all the difficulty lives the model's two smallest, most different-colored boxes are both sitting in the row that matters most to the business.

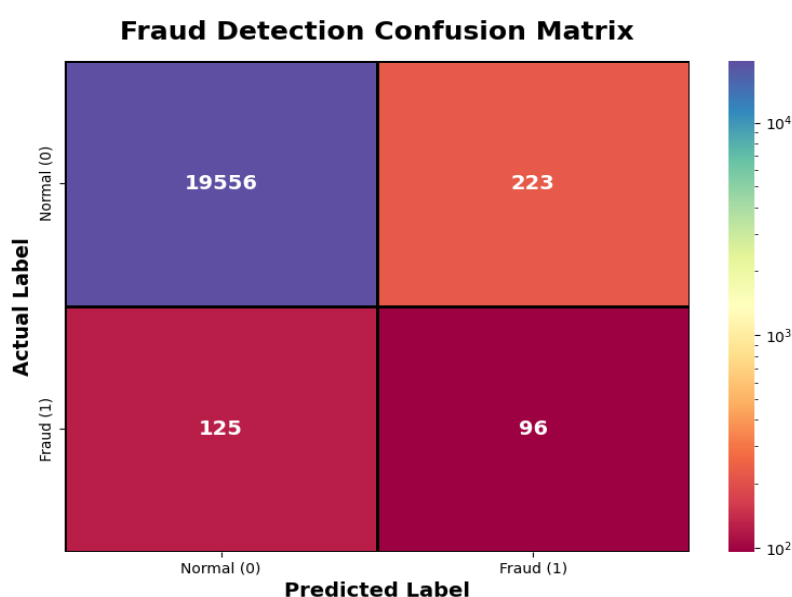


Figure 1: Confusion Matrix heatmap on a logarithmic color scale, showing all four outcome categories in clearly distinct colors.

Step 5: Analyzing Feature Importance

What is this code?

This cell reads the `.feature_importances_` attribute straight off the trained Random Forest a score, generated automatically during training, showing how much each individual column actually contributed to the trees' decisions. It matches those scores back up to the real column names, sorts them from most to least important, and plots the top 10 as a horizontal bar chart.

Why are we doing this, and why now?

A model that works but can't explain itself is a hard sell to a business, and it's also a genuine risk if a model turns out to be secretly relying on something it shouldn't (like a customer's raw account number), that's a sign of data leakage that needs fixing before it ever goes near production. This cell exists to open

the black box and check the model's actual reasoning, right after seeing that its raw performance wasn't perfect.

What does this chart actually show?

One bar towers over every other bar on the chart: `amt`, the transaction amount, sitting at roughly 0.50 meaning transaction amount alone accounts for about half of everything this model bases its decisions on. That's a direct, satisfying confirmation of exactly what Week 2's boxplots and KDE curves hinted at with soft, visual evidence amount isn't just correlated with fraud, it's genuinely the single strongest signal the model found. Everything else trails far behind in a tightly clustered group around 0.02 - 0.05: `trans_date_is_weekend`, `trans_time_day`, `trans_time_is_night`, `customer_num_trans_1_day`, `trans_time_hrs`, `city_pop`, and three of the `merchant_risk_X_day` columns. And here's the reassuring answer to the question Step 1 raised: none of the raw identifier-style columns (`cc_num`, `zip`, `acct_num`, `lat`, `long`) made it anywhere near the top 10, even though they were technically available to the model. In plain terms, the model taught itself to focus on genuine behavioral and timing patterns rather than memorizing specific accounts or locations which is exactly the kind of thing you want to see, even though tidying those ID columns out in a future pass is still good practice.

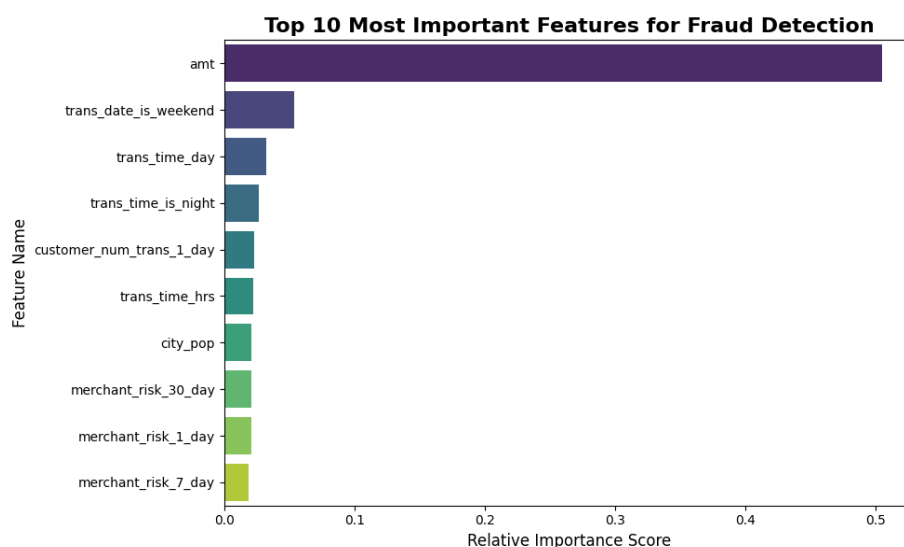


Figure 2: Top 10 most important features driving the Random Forest's fraud decisions.

Step 6: The ROC Curve and AUC Score

What is this code?

This cell pulls the model's raw fraud probability for every test transaction (not just its final yes/no answer) using `model.predict_proba()`, then uses that to calculate a Receiver Operating Characteristic (ROC) curve and its Area Under the Curve (AUC) score, and plots it.

Why are we doing this, and why now?

Step 3's confusion matrix only tells us how the model performs at one single decision threshold the default 50% cutoff. The ROC curve answers a bigger question: across every possible threshold from 0%

to 100%, how good is this model at ranking a genuine fraud case as riskier than a genuine normal case? AUC boils that entire curve down into one clean number between 0 and 1, where 0.5 means the model is no better than a coin flip, and 1.0 means it's perfect.

What does this chart actually show?

The orange curve climbs steeply up the left edge of the chart before bending toward the top right, sitting well above the diagonal dashed “random guess” line the entire way. The printed AUC score is 0.8926. That's a genuinely strong number, it means that if you handed this model one real fraud case and one real normal case at random, it would correctly rank the fraud case as riskier about 89% of the time. This is actually a really important, reassuring counterbalance to Step 3's more sobering precision/recall numbers: the model's underlying sense of risk is quite good, and a lot of the disappointing precision and recall we saw earlier comes down to exactly where the decision threshold was drawn, not the model being fundamentally confused.

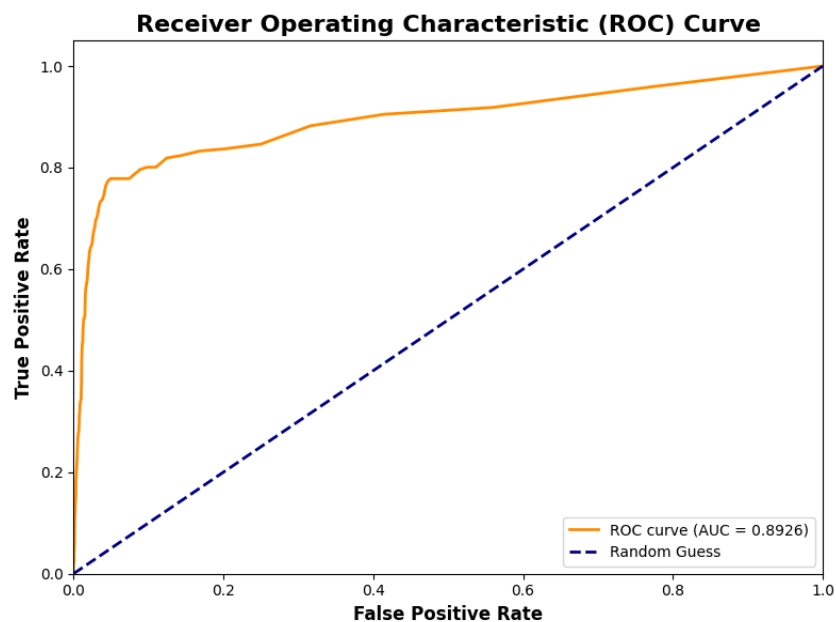


Figure 3: ROC Curve, AUC = 0.8926, a strong score showing the model ranks fraud risk well across every possible threshold.

Step 7: The Precision-Recall Curve

What is this code?

This cell calculates and plots a Precision-Recall (PR) curve, along with a single summary number called Average Precision (AP), using the same predicted probabilities from Step 6.

Why are we doing this, and why now?

The ROC curve is a great general-purpose tool, but it can actually be a little too optimistic-looking on severely imbalanced problems like this one, because it factors in the enormous pool of correctly-handled normal transactions. The Precision-Recall curve deliberately ignores that comfort and focuses entirely on

the minority class fraud giving a more realistic, more demanding picture of performance exactly where it matters most for the business.

What does this chart actually show?

The purple curve starts near the top-left at very low recall, dips down quickly, and then spends most of its length hovering in the 0.25 - 0.40 precision range as recall climbs, before trailing off toward the bottom-right. The Average Precision score is 0.2500. In plain terms, averaged across every possible way you could set the alert threshold, only about 1 in 4 fraud alerts this model raises would actually be correct. Combined with Step 6's strong AUC score, the honest picture Week 3 leaves us with is this: the model's underlying instincts about risk are genuinely good, but its current default settings and the sheer rarity of fraud in the real world mean a real deployment would need careful threshold-tuning (and, as Step 9 shows, a second model working alongside it) before it's ready to make unsupervised decisions on its own.

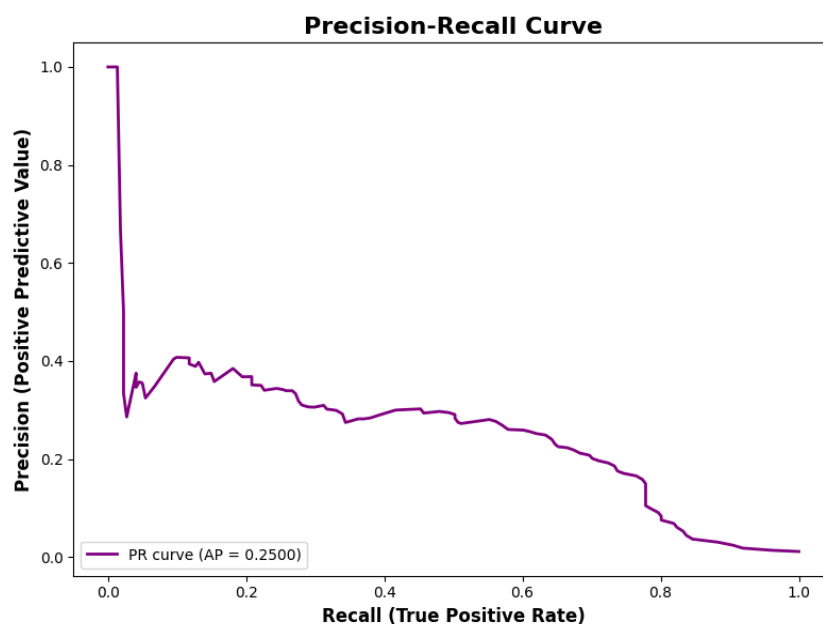


Figure 4: Precision-Recall Curve, Average Precision = 0.2500, the more demanding, fraud-focused view of performance.

Step 8: Saving the Trained Model for Production

What is this code?

This cell uses `joblib.dump()` to serialize the entire trained Random Forest, every one of its 100 decision trees, fully intact into a single file on disk called `random_forest_fraud_model.pkl`, then double-checks the file actually exists and reports its size.

Why are we doing this, and why now?

Training this model took real time and computing effort. Nobody wants to retrain 100 decision trees from scratch every single time a new transaction needs to be checked that would make the system painfully slow and completely impractical for real-time use. Saving the trained model to disk freezes its current

“learned” state permanently, so it can be loaded back into memory in a couple of seconds whenever it's needed, whether that's tomorrow or six months from now.

What did we get?

The console confirmed: “SUCCESS! Model saved successfully as 'random_forest_fraud_model.pkl'.” with a File Size of 57.54 MB, followed by “Week 3 is now complete. The model is ready for deployment in Week 4!” This is the exact file that gets loaded and put to work in the Week 4 section of this report it's the bridge between the two weeks.

Step 9: Unsupervised Anomaly Detection with Isolation Forest

What is this code?

This final Week 3 cell trains a second, completely different kind of model: an Isolation Forest, initialized with `IsolationForest(n_estimators=100, contamination=0.01, random_state=42, n_jobs=-1)`. Critically, it's trained with `isolation_model.fit(X_train)` notice there's no `y_train` passed in at all. It's then run against the test set to get predictions (-1 for anomaly, 1 for normal) and raw anomaly scores, and finally saved to disk the same way the Random Forest was.

Why are we doing this, and why now?

The Random Forest we just built and graded is a supervised model it only knows how to recognize fraud patterns it has already seen labeled examples of in the training data. That's a real weakness: a genuinely new fraud tactic that's never appeared in the historical data before would sail straight past it. Isolation Forest works completely differently it doesn't use fraud/normal labels at all. Instead, it randomly slices the data apart over and over, and anything that gets isolated into its own little slice unusually quickly is flagged as an outlier, purely because it looks statistically strange compared to everything else, whether or not the model has ever seen that specific pattern before. The parameters matter here: `n_estimators=100` builds 100 random isolation trees, `contamination=0.01` tells the model to assume roughly 1% of the data is anomalous (matching our known ~1.1% fraud rate), `random_state=42` keeps results reproducible, and `n_jobs= -1` uses every available CPU core to speed up training.

What did we get, and what does it mean?

The console reported: Total Test Transactions Evaluated: 20,000, and Anomalies Flagged by Isolation Forest: 330, followed by confirmation that `isolation_forest_fraud_model.pkl` was saved successfully at 1.47 MB. Worth noting honestly: 330 out of 20,000 works out to about 1.65% of the test set being flagged, a little higher than the 1% contamination assumption baked into the model a small, normal side effect of exactly where the model's internal scoring boundary happens to fall, not an error. The real value of this step isn't a single clean accuracy number; it's that we now have a second, entirely independent way of spotting suspicious activity, one that looks for statistical strangeness rather than memorized patterns. Week 4 leans on this directly, combining both models' opinions so that a weakness in one can be caught by the other.

Week 3 Summary

By the end of Week 3, the balanced training data from Week 2 had actually been put to work. A Random Forest was trained and honestly graded against a completely untouched test set: 98% overall accuracy, but a much more telling 43% recall and 30% precision on the fraud class specifically, alongside a genuinely strong 0.8926 ROC-AUC score meaning the model's underlying sense of risk is good, even if its current threshold setting leaves real room for improvement. Feature importance confirmed transaction amount as the single dominant signal, matching everything Week 2's charts had already hinted at, and reassuringly showed the model wasn't leaning on raw identifier columns despite having access to them. And to cover the Random Forest's biggest weakness its blindness to fraud patterns it has never seen before a second, unsupervised Isolation Forest model was trained and saved alongside it. Both trained models were written to disk, ready to be picked up and put into a real, live system in Week 4.

WEEK 4: Model Deployment

Week 4 is where the two saved model files stop being notebook artifacts and become an actual, callable system. There are two pieces of code involved `app.py` (the live API) and `test_api.py` (the test suite that proves it works), plus a genuinely valuable third piece of evidence: the real terminal output and the real email alert from an actual run of the system, which we'll walk through in full.

Cell 1 (`app.py`): The Flask API: Full Walkthrough

This is the heart of Week 4, a live web server that can receive one transaction at a time and return an instant fraud decision. It's substantial enough that it's worth walking through it in the same numbered sections the code itself is already organized into.

Section 1: Imports and Setup

The script imports Flask, request, and jsonify from the Flask framework, joblib to load the saved models, pandas to structure incoming data, and boto3 (the official AWS SDK) to talk to Amazon's cloud. `app = Flask(__name__)` then initializes the actual web application object that everything else attaches to.

In plain terms: Flask is what turns an ordinary Python script into something that can sit and listen on the internet (or a local network) for incoming requests, the way a tiny web server would. Without it, this code could still make predictions, but nothing outside the script could ever ask it to.

Section 2: AWS SNS Configuration (The Alert System)

This section sets `SNS_TOPIC_ARN` and `AWS_REGION` (pointing at a specific pre-created "FraudAlertTopic" in AWS's eu-north-1 region), along with an `ACCESS_KEY` and `SECRET_KEY` used to authenticate with AWS. It then tries to create a boto3 SNS client using those credentials, setting a flag called `aws_ready` to True if the connection succeeds, or False (with a printed warning, but no crash) if it fails.

Why build it this way: the try/except wrapper here is a deliberate, sensible safety design. If the AWS credentials ever expire, or the network is down, or the SNS topic gets deleted, the entire fraud-detection

API doesn't crash and stop working, it simply loses its ability to send email alerts while continuing to detect and block suspicious transactions perfectly normally. That's a genuinely good piece of engineering: the core safety function (blocking fraud) is never allowed to depend on the health of a secondary feature (emailing someone about it).

```
ACCESS_KEY = "AKIA*****ROD"    (masked - see app.py for the real value)
SECRET_KEY = "50H9*****Gya9"   (masked - see app.py for the real value)
```

A quick, practical flag for the team, separate from how the code behaves: the real access key and secret key are currently hardcoded in plain text directly inside app.py. That's fine for a local demo, but since this file and this report are being shared around the team, it's worth moving these two values into environment variables (or AWS Secrets Manager) before this goes anywhere near a shared repository or a production server, and rotating this particular key afterward. This doesn't affect anything about how the system works; it's purely a housekeeping note.

Section 3: Loading the Machine Learning Models

This section loads both trained models from Week 3 straight into memory: `rf_model = joblib.load('random_forest_fraud_model.pkl')` and `iso_model = joblib.load('isolation_forest_fraud_model.pkl')`, with print statements confirming success on either side.

The comment in the code itself explains the reasoning well, and it's worth repeating plainly: these two files are loaded here, once, at the very top of the script, outside of the function that actually handles incoming requests. If they were loaded inside that function instead, the server would have to reload a 57-megabyte file and a 1.5-megabyte file from disk on every single incoming transaction, which would make the whole system unusably slow. Loading once at startup keeps every individual prediction fast.

Section 4: The /predict Endpoint (The Decision Engine)

This is the actual route “`@app.route('/predict', methods=['POST'])`” that anything wanting a fraud decision has to call. It's built in five clearly labeled steps.

- Step A = Data Ingestion: `request.get_json()` pulls the incoming transaction data out of the web request, and `pd.DataFrame([data])` wraps that single transaction into a one-row table, because that's the exact shape both models expect to receive.
- Step B = Model Evaluation: the same transaction is run through both models to get three separate readings `rf_prediction` (the Random Forest's plain 0-or-1 answer), `rf_proba` (its exact statistical confidence, turned into a `risk_score` percentage), and `iso_prediction` (the Isolation Forest's -1-or-1 anomaly flag, turned into an `is_anomaly` true/false).
- Step C = The Business Logic: if `rf_prediction == 1` OR `is_anomaly` OR `risk_score > 50.0`, the transaction is marked `BLOCKED_SUSPICIOUS`; otherwise it's `APPROVED`. This is a deliberately cautious, “any single red flag trips the alarm” rule, it means the two models don't have to agree with each other to catch something; either one raising a hand is enough.

- Step D = AWS Cloud Alerting: this block only runs at all if the decision just came back `BLOCKED_SUSPICIOUS` and the AWS connection is healthy. It builds a clean, readable alert message with the transaction amount, account number, risk score, anomaly flag, and decision, then publishes it to the SNS topic, which is what actually triggers a real email.
- Step E = Returning the Result: the endpoint sends back a structured JSON response containing the final decision, both models' evaluation metrics, and whether an alert was actually sent, including AWS's own message ID if it was.

There's also a try/except wrapped around the entire function, so if the incoming data is ever malformed or missing a field the models need, the API returns a clean JSON error message with a 400 status code instead of crashing the whole server a normal, sensible safety net for anything exposed to the outside world.

The very last line, `app.run(host='0.0.0.0', port=5000, debug=True)`, starts the server. Using `host='0.0.0.0'` means the app listens on every network interface on the machine, not just its own local loopback which is exactly why, in the real run below, the server was reachable at both `127.0.0.1:5000` and `192.168.1.6:5000` at the same time. `debug=True` is genuinely useful during development (it gives detailed error pages and automatically restarts the server when the code changes), but it's worth flagging as something to switch off before this ever runs in a real production environment, since debug mode can leak internal details if something goes wrong.

Cell 2 (`test_api.py`): The Testing Suite

What is this code?

This script plays the role of a real credit card terminal talking to the API. It loads `X_test_final.csv`, grabs the very first row as a genuine, everyday transaction, and sends it to the running API. Then it takes a copy of that same row and deliberately manipulates it into an obvious fraud scenario before sending that too.

Why are we doing this, and why now?

A live API is only proven to work once you've actually thrown real requests at it and checked the responses. This script exists purely to demonstrate, end to end, that the system can correctly tell the difference between an everyday purchase and something dangerous, which is the entire point of everything built in Weeks 1 through 4.

What exactly does the fraud scenario change, and why those specific fields?

Scenario 2 doesn't just bump up the amount, it stacks several red flags on top of each other on purpose. `amt` is set to `$99,999.00` (there's even a commented-out earlier attempt at `$45,000.00` still visible in the code, showing this number was deliberately tuned upward until it reliably tripped the system). Alongside that, `customer_num_trans_1_day`, `_7_day`, and `_30_day` are pushed up to 500, 1500, and 3000 simulating a customer who suddenly transacted hundreds of times in a single day, a classic velocity-fraud signature. All four `merchant_risk_X_day` columns are set to 100.0, simulating a merchant with a maxed-out historical risk score. `customer_avg_amout_1_day` is set to match the huge amount, and

trans_time_is_night is forced to 1.0, simulating a purchase made in the middle of the night directly echoing the late-night fraud spike Week 2 discovered. It's a genuinely well-constructed stress test, combining amount, velocity, merchant risk, and timing all at once.

The Live Proof: Actually Running the System

This is the part of Week 4 that turns all of the above from a plan into a demonstrated fact. Here's exactly what happened when app.py and test_api.py were actually run, back to back, on a real machine.

Starting the server

Running python app.py printed a clean startup sequence: the AWS SNS client connected successfully, both models loaded into memory, and the Flask server came online, listening on both http://127.0.0.1:5000 and http://192.168.1.6:5000 as expected from the host='0.0.0.0' setting.

Four identical warnings appeared during model loading InconsistentVersionWarning, noting that the models were originally trained and saved using scikit-learn version 1.6.1, but are now being loaded on a machine running version 1.9.0. This isn't a crash and the models still loaded and worked correctly, but it's worth flagging as a piece of housekeeping: for a production deployment, it's good practice to either pin the exact scikit-learn version used in Week 3 inside the deployment environment, or simply retrain and re-save the models using the same environment they'll actually run in, so this warning disappears entirely rather than being something the team has to consciously look past every time the server starts.

```
SUCCESS: AWS SNS Client connected. Real-time alerting is ACTIVE.
Loading Random Forest and Isolation Forest models...
SUCCESS: Models loaded into memory.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.6:5000
```

Sending the two test transactions

Running python test_api.py right after produced two very telling results.

Scenario 1, the untouched normal transaction, came back with a supervised_risk_score_percentage of exactly 3.0%, unsupervised_anomaly_detected of false, a final decision of APPROVED, and, correctly, no email was sent (sns_notification_sent: false). Exactly what a genuine, everyday purchase should produce — quiet, unremarkable, no false alarm.

Scenario 2, the manipulated \$99,999 fraud attempt, came back with a supervised_risk_score_percentage of just 22.0%, unsupervised_anomaly_detected of true, a final decision of BLOCKED_SUSPICIOUS, and sns_notification_sent: true, with a real AWS message ID of eb29d5cf-2690-5006-8550-b6edcb51a30c confirming an alert genuinely went out.

```
--- API RESPONSE FOR FRAUD TRANSACTION ---
{
  "decision": "BLOCKED_SUSPICIOUS",
  "evaluation_metrics": {
    "supervised_risk_score_percentage": 22.0,
    "unsupervised_anomaly_detected": true
  }
}
```

```
},
"cloud_alert_system": {
  "sns_notification_sent": true,
  "aws_message_id": "eb29d5cf-2690-5006-8550-b6edcb51a30c"
}
}
```

The one detail in this run that's genuinely worth pointing out to the team

Look closely at that fraud scenario result: the Random Forest's own risk score was only 22% nowhere near the 50% threshold that would trigger a block on its own, and consistent with Step 3's finding back in Week 3 that this model's precision and recall on fraud, taken alone, are still fairly weak. If the system relied on the Random Forest by itself, this heavily manipulated, obviously suspicious transaction would have sailed through as APPROVED. It only got caught because the Isolation Forest independently flagged it as an anomaly (unsupervised_anomaly_detected: true), which tripped the OR-based rule in Step C of the /predict endpoint. This is, in a very real and concrete sense, exactly why Week 3 trained a second model in the first place and this live run is the actual proof that the two-model design works the way it was meant to, catching something the first model alone would have missed.

The real email proof the alert genuinely reached an inbox

The AWS message ID printed in the terminal is good evidence the alert was sent, but the screenshot below is the actual result a real email, delivered to a real inbox, generated automatically the moment the fraud transaction was blocked. Every field in the email matches exactly what the API returned: the \$99,999.00 amount, the account number, the 22.0% risk score, the anomaly flag, and the BLOCKED_SUSPICIOUS decision. This confirms the entire chain genuinely works end to end not just “the code runs without errors,” but the full path from a suspicious transaction, through both models, through the business logic, through AWS SNS, and all the way into a human being's actual inbox.

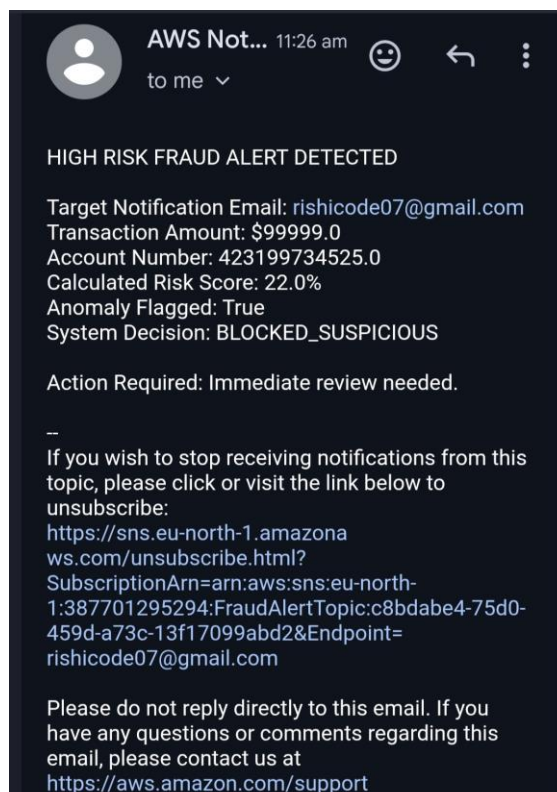


Figure 5: The actual AWS SNS email alert received on a phone, confirming the fraud alert pipeline works end to end.

Week 4 Summary

Week 4 took the two models Week 3 trained and evaluated and turned them into something real. A Flask API was built that loads both models once at startup and makes a combined decision on every incoming transaction, using a deliberately cautious rule where either model raising a flag is enough to block something and trigger a real email alert through AWS SNS. That system wasn't just written it was actually run, and the results prove the design works: a genuine test transaction sailed through as APPROVED with a 3% risk score, while a heavily manipulated fraud attempt was correctly caught and alerted on, specifically because the Isolation Forest picked up on an anomaly the Random Forest's own score alone would have missed. A real email landed in a real inbox as proof that the entire chain model, decision logic, and cloud alerting genuinely works end to end.

Overall Summary: Weeks 1 Through 4

Stepping all the way back: this project started as a raw, unexamined database table. Week 2 turned it into a dataset the team genuinely understood knowing exactly how rare fraud is, what it tends to look like, and how to fairly balance it without cheating the evaluation. Week 3 turned that understanding into two trained, honestly-graded models a supervised Random Forest that's strong at ranking overall risk (0.89 AUC) but imperfect at a single fixed threshold (43% recall, 30% precision), paired with an unsupervised Isolation Forest built specifically to catch what the first model can't. Week 4 turned those two models into a real, live, tested system one that was actually run, actually caught a simulated fraud attempt the first model alone would have missed, and actually sent a real alert to a real inbox.

The honest throughline across all four weeks is worth saying plainly to the team: every step was validated with real numbers rather than assumptions, every shortcut that could have quietly inflated the results (like leaking test data or applying SMOTE too early) was deliberately avoided, and the project's current, true state is a working proof of concept with clearly identified next steps tuning the decision threshold, cleaning out the leftover ID-style columns, and resolving the scikit-learn version mismatch rather than a finished, polished product. That's exactly the kind of honest status a team should walk away from this report with.